

Uniswap Protocol Fees on BNB, Celo, and Polygon Audit



Uniswap

May 15, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Governance Proposal 4	5
Wormhole's Native Token Transfer (NTT)	5
Security Model and Trust Assumptions	6
High Severity	7
H-01 Polygon Deployment JSON Loaded From BNB Path	7
Low Severity	8
L-01 WormholeReleaser Quotes Wormhole Core Fee Instead of NttManager Delivery Price	8
L-02 ExchangeReleaser Nonce Does Not Bind Assets, Recipient, or Threshold	9
Notes & Additional Information	10
N-01 Unused CELO_PORTAL Constant In Constants.sol	10
N-02 Missing Security Contact	10
N-03 TODO Placeholder In Governance Proposal Description	11
N-04 Polygon WormholeTransceiver Pauser Is Not Renounced	11
N-05 Incorrect NatSpec And Inline Comments	12
N-06 WormholeReleaser Refund Allows Front-Runners to Steal Pre-Funded Ether	13
Conclusion	15
Appendix	16
Issue Classification	16

Summary

Type	DeFi	Total Issues	9 (7 resolved, 1 partially resolved)
Timeline	From 2026-05-05 To 2026-05-11	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	1 (1 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	2 (1 resolved)
		Notes & Additional Information	6 (5 resolved, 1 partially resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the [Uniswap/protocol-fees](#) repository at the [a3a374f](#) commit from [PR #130](#).

In scope were the following files:

```
protocol-fees
├── script
│   └── proposal-4
│       ├── ActivateL2sProposal.s.sol
│       ├── Constants.sol
│       ├── Interfaces.sol
│       └── deploys
│           ├── ConfigWormholeInfraBNBChain.s.sol
│           ├── ConfigWormholeInfraEthereum.s.sol
│           ├── ConfigWormholeInfraPolygon.s.sol
│           ├── DeployAndConfigureFeeInfraBNBChain.s.sol
│           ├── DeployAndConfigureFeeInfraPolygon.s.sol
│           ├── DeployWormholeInfraBNBChain.s.sol
│           ├── DeployWormholeInfraEthereum.s.sol
│           └── DeployWormholeInfraPolygon.s.sol
└── src
    ├── interfaces
    │   └── wormhole
    │       ├── INttManager.sol
    │       ├── INttToken.sol
    │       └── IWormhole.sol
    ├── releasers
    │   └── WormholeReleaser.sol
    └── wormhole
        └── SyntheticNttUni.sol
```

System Overview

Governance Proposal 4

The audited code corresponds to Uniswap Governance Proposal 4, which activates the protocol fee switch for the Uniswap V2 and V3 deployments on Celo, BNB Chain, and Polygon. Although the Uniswap protocol has accrued swap volume on these networks for some time, no infrastructure existed to collect protocol fees, convert them into UNI, and ultimately burn that UNI on Ethereum, where the canonical UNI token resides. Proposal 4 introduces the contracts and deployment scripts required to close this gap and extends the existing fee collection system to these three chains.

The collection layer follows the same architecture already deployed on the OP Stack networks. On each foreign chain, a TokenJar contract acts as a universal sink that receives protocol fees from the Uniswap V2 and V3 factories. The V2 factory pushes liquidity provider tokens directly into the TokenJar via the `feeTo` setting, while a `V3OpenFeeAdapter` contract owns the V3 factory and exposes permissionless functions that allow anyone to trigger fee collection into the TokenJar. The TokenJar itself is intentionally minimal and only allows a single configured releaser address to withdraw the accumulated balances atomically.

Wormhole's Native Token Transfer (NTT)

Converting these accumulated fees into a UNI burn requires bridging logic, since neither BNB Chain nor Polygon hosts the canonical UNI token. To solve this, the proposal adopts Wormhole's Native Token Transfer (NTT) system under a hub-and-spoke model. Ethereum acts as the hub, where the canonical UNI is locked inside an NttManager contract operating in LOCKING mode. BNB Chain and Polygon act as spokes, where a synthetic UNI token (SyntheticNttUni, symbol NUNI) is minted and burned by a local NttManager operating in BURNING mode. Cross-chain messages between these managers are carried by a WormholeTransceiver contract on each chain that wraps the Wormhole core bridge.

The end-to-end fee burn flow on a foreign chain proceeds through the following sequence of actions:

1. A searcher competing for the protocol fees deposits a fixed threshold amount of synthetic UNI into the local WormholeReleaser.

2. The WormholeReleaser instructs the TokenJar to release its accumulated assets to a recipient designated by the searcher.
3. The WormholeReleaser approves the local NttManager to spend the deposited synthetic UNI and triggers a transfer toward Ethereum with the burn address as recipient.
4. The local NttManager burns the synthetic UNI on the foreign chain and forwards the transfer instruction to the WormholeTransceiver.
5. The WormholeTransceiver publishes a message through the Wormhole core bridge, which is then attested by the Wormhole Guardians.
6. On Ethereum, the canonical NttManager consumes the attested message, unlocks the equivalent amount of UNI, and forwards it to the dead address, completing the burn.

Security Model and Trust Assumptions

Throughout the codebase, the following trust assumptions were identified:

- Wormhole is trusted to be configured and operated correctly.
- Ownership and privileged roles across the per-chain infrastructure are governance-controlled:
 - `TokenJar`, `WormholeReleaser`, `SyntheticNttUni`, and `V3OpenFeeAdapter` each expose an owner (and, where applicable, a `thresholdSetter` or `feeSetter`) that controls configuration on each foreign chain.
 - The Wormhole NTT Manager and Transceiver are deployed behind upgradeable proxies whose ownership cascades from the manager.
 - Deployment and configuration scripts transfer all of these roles to the per-chain governance receiver: the `Timelock` on Ethereum mainnet, `UNISWAP_WORMHOLE_MESSAGE_RECEIVER` on BNB and Celo, and the `EthereumProxy` on Polygon.
 - The Uniswap governance flow ultimately controls upgrades, releaser swaps, threshold updates, and fee configuration on each chain.
- The `release` function on `WormholeReleaser` is permissionless and not a privileged role.

High Severity

H-01 Polygon Deployment JSON Loaded From BNB Path

In `ActivateL2sProposal.s.sol`, `_loadDeployments` reads the Polygon deployment artifact from `BNB_DEPLOY_PATH` instead of `POLYGON_DEPLOY_PATH` (declared at lines 22–23 but never used).

As a result, `polygonTokenJar` and `polygonOpenV3FeeAdapter` are populated with BNB Chain addresses, and STEP 3 forwards them via `IPolygonFxRoot.sendMessageToChild` to set Polygon's V2 `feeTo` and transfer ownership of Polygon's V3 factory to addresses that are unlikely to contain the corresponding contracts on Polygon. V2 fees would accrue to a dead address, and ownership of the Polygon V3 factory would be lost to a non-`V3OpenFeeAdapter` address.

The same defect pattern is duplicated on [line 74](#) in `SimulateExecution.s.sol`, where `_loadDeployments` also reads `BNB_DEPLOY_PATH` for the Polygon load.

Consider replacing `BNB_DEPLOY_PATH` with `POLYGON_DEPLOY_PATH` on line 369, and adding a sanity check that the BNB and Polygon loaded addresses differ to prevent this class of copy-paste error.

Update: Resolved at commit [0e12a7c](#). The team stated:

Path parameter was fixed. A `require` check was added such that the JSON data corresponding to BNB Chain and Polygon deployments are not equal.

The check was applied to the JSON data instead of the addresses (as recommended) to avoid a case where the contract addresses on Polygon and BNB Chain may match. Since the deployment JSON data includes transaction hashes, the total JSON contents will never match.

Low Severity

L-01 WormholeReleaser Quotes Wormhole Core Fee Instead of NttManager Delivery Price

Inside `WormholeReleaser._afterRelease`, the value forwarded to `NTT_MANAGER.transfer` is `read` directly from the Wormhole core bridge:

```
uint256 messageFee = WORMHOLE.messageFee();
...
NTT_MANAGER.transfer{value: messageFee}({
    amount: amount, recipientChain: WORMHOLE_DEFINED_ETH_CHAIN_ID, recipient: BURN_ADDRESS
});
```

However, `NttManager.transfer` does not consume the raw Wormhole core fee. It `iterates` over every enabled transceiver and requires the caller to supply the sum of each transceiver's `quoteDeliveryPrice` for the destination chain. With the current deployment the only registered transceiver is a pure `WormholeTransceiver`, whose quoted price happens to equal `WORMHOLE.messageFee()`, so the call succeeds today.

The `NttManager` owner retains the ability to register additional transceivers via `setTransceiver`. If a second transceiver is enabled, every subsequent `release` reverts inside `_afterRelease` because the supplied `value` undershoots the manager's required total. The releaser has no upgrade path, so the entire fee-collection pipeline on that chain is bricked until governance either removes the extra transceiver or redeploys the releaser.

Consider quoting the delivery price from the `NttManager` itself by querying the `quoteDeliveryPrice` function or the equivalent helper exposed by the deployed NTT version, and forwarding that value to `NTT_MANAGER.transfer`.

Update: Resolved at commit [1fffcc5](#). The team stated:

Replaced the `WORMHOLE.messageFee` query with `NTT_MANAGER.quoteDeliveryPrice` to handle multiple transceivers.

Documentation was updated to reflect this change. A test case to handle multiple transceivers was added, as were some updates to mock contracts in testing.

NOTE: The `messageFee` query was the only reason the `WormholeReleaser` needed to be aware of the wormhole core bridge. As a result, we removed the wormhole core

contract as a constructor parameter and that change propagated out to tests, scripts, code comments, and documentation. All of these changes are encapsulated in this commit.

L-02 ExchangeReleaser Nonce Does Not Bind Assets, Recipient, or Threshold

The `handleNonce` modifier used to gate `ExchangeReleaser.release` enforces a strict sequence on invocations, and its natspec `advertises` this as front-running protection. In practice, the modifier only enforces ordering, but it does not commit the caller to the `assets` array, the `recipient`, or the `threshold` in force at submission time. An observer in the mempool can replay the same nonce with a different recipient, an expanded asset list, or under a freshly changed threshold, capturing value the original searcher intended to claim and reverting their transaction with `InvalidNonce`.

Consider rewording the `Nonce` Natspec to reflect that it provides ordering and replay protection only, not front-running protection. For stronger guarantees, consider binding the nonce to a commitment over the release parameters so that the slot cannot be reused with different terms.

Update: Acknowledged, not resolved. The team stated:

Acknowledged. The `ExchangeReleaser` is upstream from the `WormholeReleaser` and other child instances of the `ExchangeReleaser` have already been deployed to other chains.

In the interest of not maintaining diverging implementations from existing deployments, we plan on leaving this as-is since this system is not permanent.

Notes & Additional Information

N-01 Unused `CELO_PORTAL` Constant In `Constants.sol`

`Ethereum.CELO_PORTAL` is declared in `Constants.sol` and validated by `smokeCheck`, but is never referenced by any deploy, configuration, or proposal script in the audit scope.

Consider removing the dead constant and its `smokeCheck` entry, or wiring it to its intended use if the omission was unintended.

Update: Resolved at commit [7815c38](#). The team stated:

| *Removed the constant and associated smoke check.*

N-02 Missing Security Contact

Providing a specific security contact (such as an email address or ENS name) within a smart contract significantly simplifies the process for individuals to communicate if they identify a vulnerability in the code. This practice is quite beneficial as it permits the code owners to dictate the communication channel for vulnerability disclosure, eliminating the risk of miscommunication or failure to report due to a lack of knowledge on how to do so. In addition, if the contract incorporates third-party libraries and a bug surfaces in those, it becomes easier for their maintainers to contact the appropriate person about the problem and provide mitigation instructions.

Throughout the codebase, multiple instances of contracts not having a security contact were identified:

- The [WormholeReleaser contract](#)
- The [SyntheticNttUni contract](#)

Consider adding a NatSpec comment containing a security contact above each contract definition. Using the `@custom:security-contact` convention is recommended as it has been adopted by the [OpenZeppelin Wizard](#) and the [ethereum-lists](#).

Update: Resolved at commit [43b6ce7](#). The team stated:

| Added security contact natspec to the contracts mentioned.

N-03 **TODO** Placeholder In Governance Proposal Description

The `PROPOSAL_DESCRIPTION` constant is the literal string "TODO" and is passed verbatim to `IGovernorBravo.propose` in the script's `run` function. If the script is executed as-is, the on-chain governance proposal carries "TODO" as its description, leaving voters without context on the proposal. The description is hashed into the proposal identifier and cannot be edited after submission.

Consider replacing the placeholder with a complete markdown description before mainnet execution.

Update: Resolved at commit [9d09ab8](#). The team stated:

| Added proper text.

N-04 Polygon WormholeTransceiver Pauser Is Not Renounced

The Ethereum and BNB Chain configuration scripts renounce the pauser role on both the `NttManagerNoRateLimiting` proxy and the `WormholeTransceiver` proxy. [The Polygon configuration script](#) renounces only the manager pauser, leaving the deployer EOA as the permanent pauser of the Polygon transceiver. The current operational impact is zero, since the pauser slot on the transceiver is effectively dead storage in this version of NTT. The omission is still worth flagging because it is inconsistent with the project's stated decentralization principle and forward-fragile against future NTT versions that introduce pausable transceiver paths.

Consider transferring the pauser capability of the Polygon `WormholeTransceiver` proxy to `address(0)` in the Polygon configuration script, with a matching post-condition assertion, and updating the corresponding section of [Index.md](#) to match the BNB and Ethereum entries.

Update: Resolved at commit [281239a](#). The team stated:

Added the `transferPauserCapability(address(0))` call to the `WormholeTransceiver` on Polygon.

Tests to verify the pauser authority was renounced were written in another commit but did not function at the time of the audit commit hash.

Though the test was not part of this commit, the test can be found here:

<https://github.com/Uniswap/protocol-fees/blob/281239a9577679cae2141481790c523dc92226fb/test/end-to-end/proposal-4/PostflightChecks.t.sol#L286>

N-05 Incorrect NatSpec And Inline Comments

The following NatSpec blocks and inline comments in the audit scope describe code that does not match the implementation:

- The `burn` function's `@param amount` in `SyntheticNttUni` reads "Amount to mint" instead of "Amount to burn", copied from `mint` directly above.
- The `_afterRelease` notice in `WormholeReleaser` warns about "a non-multiple of `1e8`", but the actual truncation factor in `wormholeTrim` is `1e10` raw wei (8-decimal Wormhole precision against 18-decimal token precision). A reader following the notice and setting `threshold = 5 * 1e8` would still produce dust.
- The `Deployment` struct doc in `ConfigWormholeInfraEthereum.s.sol` reads `SytheticNttUni`, instead of `SyntheticNttUni`, missing an `n`.
- The inline comment above `loadDeployments` in `ConfigWormholeInfraEthereum.s.sol` claims the function loads "BNB Chain, and ETH deployments", but it actually loads three chains (BNB, Polygon, ETH).
- Five comment blocks in `DeployAndConfigureFeeInfraPolygon.s.sol` describe the new owner, threshold-setter, or fee-setter as `UniswapWormholeMessageReceiver` ("Governance-owned Wormhole message receiver"), but the code passes `Constants.Polygon.ETHEREUM_PROXY`, the FxRoot/FxChild governance receiver. Polygon does not use Wormhole for governance dispatch. Affected lines: [L105](#), [L109](#), [L116](#), [L120](#), [L127](#), [L131](#), [L210](#), [L214](#), [L221](#), [L225](#).
- Three constants in `Constants.sol` cite prior-proposal scripts at paths that do not exist:
- `Ethereum.WORMHOLE_SENDER` cites `script/proposal-2/05_ActivateL2sProposal.s.sol`. The real path is `script/proposal-2/05_ActivateOPBaseArbProposal.s.sol`.

- `Celo.V3_OPEN_FEE_ADAPTER` cites `script/proposal-2/06_ActivateL2sProposal.s.sol`, but no `06_*` file exists in `proposal-2/`. The address is actually defined in `script/proposal-3/06_ActivateL2sProposal.s.sol` as `CELO_FEE_ADAPTER`.
- `Celo.CROSS_CHAIN_ACCOUNT` cites `script/proposal-2/05_ActivateL2sProposal.s.sol`. The real path is `script/proposal-2/05_ActivateOPBaseArbProposal.s.sol`.

Consider correcting each of these comments to match the corresponding code. For the `Constants.sol` citations specifically, consider linking to the on-chain deployment transactions instead of in-repo script paths, which can drift across refactors.

Update: Resolved at commit [7f721d8](#). The team stated:

Fixed each developer documentation issue.

N-06 WormholeReleaser Refund Allows Front-Runners to Steal Pre-Funded Ether

The `release` function in `ExchangeReleaser` is not declared `payable`, so callers cannot atomically attach `msg.value` to cover the Wormhole `messageFee` consumed inside `WormholeReleaser._afterRelease`. The only way to fund the message fee is by sending Ether to the releaser ahead of time through the `receive` fallback, which records no per-caller accounting.

At the end of `_afterRelease`, the contract forwards the entire residual balance to `msg.sender`:

```
if (address(this).balance > 0) {
    (bool success,) = msg.sender.call{value: address(this).balance}(new bytes(0));
    require(success, SenderRefundFailed());
}
```

Because the refund is keyed on `msg.sender` rather than on funds the caller actually contributed in the same transaction, any Ether sitting in the contract is claimable by whoever calls `release` next.

This enables a deterministic front-running attack:

1. A searcher prefunds the contract with `messageFee` Ether in transaction `t1`
2. Next, the searcher submits `release()` in transaction `t2`;

3. An observer front-runs `t2` with their own `release()`, paying the same `threshold` of `RESOURCE`, but pocketing the prefunded Ether via the refund and getting the asset payout.
4. The original searcher's transaction then reverts with `InvalidNonce`.

The impact is bounded today on Polygon because `WORMHOLE.messageFee()` currently returns `0` there, but Wormhole governance can raise the fee on any deployed chain at any time without redeploying the releaser, at which point the attack becomes immediately profitable on every chain where this contract is live.

Although the code comments recommend performing this operation atomically, users often do not read the entire code before interacting with it, so performing the ETH transfer and the `release` call in sequence is still something that can happen.

Therefore, consider making `release payable`, so the user doesn't need to transfer ETH before calling `release`, to prevent front-running.

Update: Partially Resolved at commit [2efca79](#). The team stated:

This design is not ideal, but unfortunately changing the release function to be payable requires a change to the parent contract, which is the parent of other contracts already deployed to other networks.

This incurs the complexity of maintaining two releaser versions based on which chains we deployed onto already versus any after this. We are planning on migrating off of this system entirely across all chains in the near term, so for now, we've added to the @dev tag a note that it must be handled atomically if the fee is non-zero.

Conclusion

Uniswap Governance Proposal 4 extends the existing protocol fee collection system to Celo, BNB Chain, and Polygon, enabling the protocol to capture fees from the Uniswap V2 and V3 deployments on those networks and ultimately burn the equivalent UNI on Ethereum. The audited code introduces a Wormhole Native Token Transfer integration that connects the foreign chain fee infrastructure to the canonical UNI token through a hub-and-spoke model, along with the deployment and configuration scripts that bring the system online and the governance proposal that activates the fee switch on each target chain.

The audit identified one high-severity issue, two low-severity issues, and several findings of Note severity. The codebase was found to be well-structured and thoroughly documented, with extensive on-chain assertions in the deployment scripts that reduce the likelihood of misconfiguration during the prerequisite phase.

The recommended fixes are self-contained and do not significantly alter the design of the proposal or the behavior of the deployed contracts, so a follow-up fix review should be sufficient and a full re-audit is not required. However, any change to the ordering of the prerequisite actions, the inclusion of additional governance actions in the proposal, or the introduction of new functions in the Wormhole integration or the SyntheticNttUni token should be subject to a new audit, as such changes can affect the cross-chain authority model and the assumptions that underpin the reviewed flow.

The Uniswap team was highly cooperative throughout the engagement and provided all the documentation, context, and clarifications needed to complete the review efficiently.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.